

Secure System Development:
Security Implementation Report
for
Sentient
(Game Application with Emerging Tech)

In Partial Fulfillment of the Requirements for the Course

INTE 302 - Information Assurance and Security 1

Course Instructor: **Mr. Cristian O. Balatbat, MIT**

Submitted By: **John Angelo Mikhail A. Reveche**

Date Submitted: **May 27, 2026**

1. System Overview

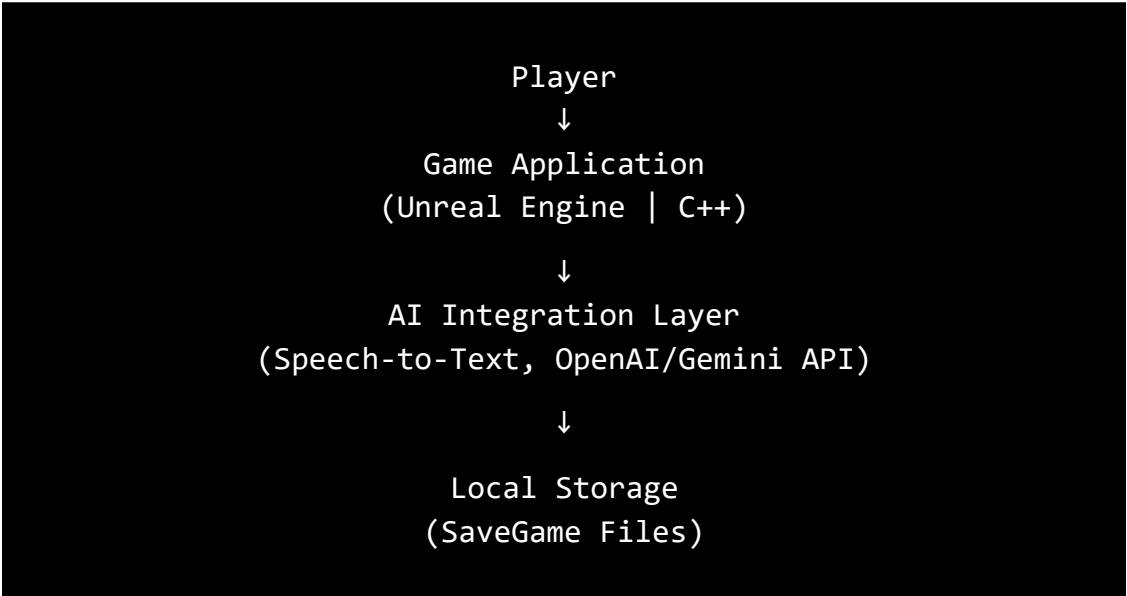
1.1 System Description

Sentient Game is a Mystery Game Application that uses Artificial Intelligence such as Gemini or OpenAI to generate dialogue responses for the NPCs to the player dynamically and allows the user to use their microphones to speak to the NPCs directly.

1.2 Target Users

- Players
- Game Developers

1.3 Basic Architecture Diagram



2. Threat & Vulnerability Identification

Threat	Possible Vulnerability	Impact
Memory Editing or Runtime Cheating	Sensitive values are stored plainly in the memory and can be scanned or modified using cheat tools.	Gaining unlimited currency, HP, resources, or stats, which causes unfair gameplay and loss of game balance.

Unauthorized Resource Injection	The game may trust unverified methods when updating the resources, such as currencies.	Generation of fake rewards, duplicate items, or manipulate progression systems.
Save File Tampering	Save files are stored locally and can be edited externally.	Altering progression, unlocking content, or corrupting game data.
Asset replacement and malicious file modification	Game assets can be unpacked and replaced if integrity verification is not present.	Modified assets or malicious content may run in the game which may affect gameplay integrity and potentially cause crashes.

3. Security Implementation

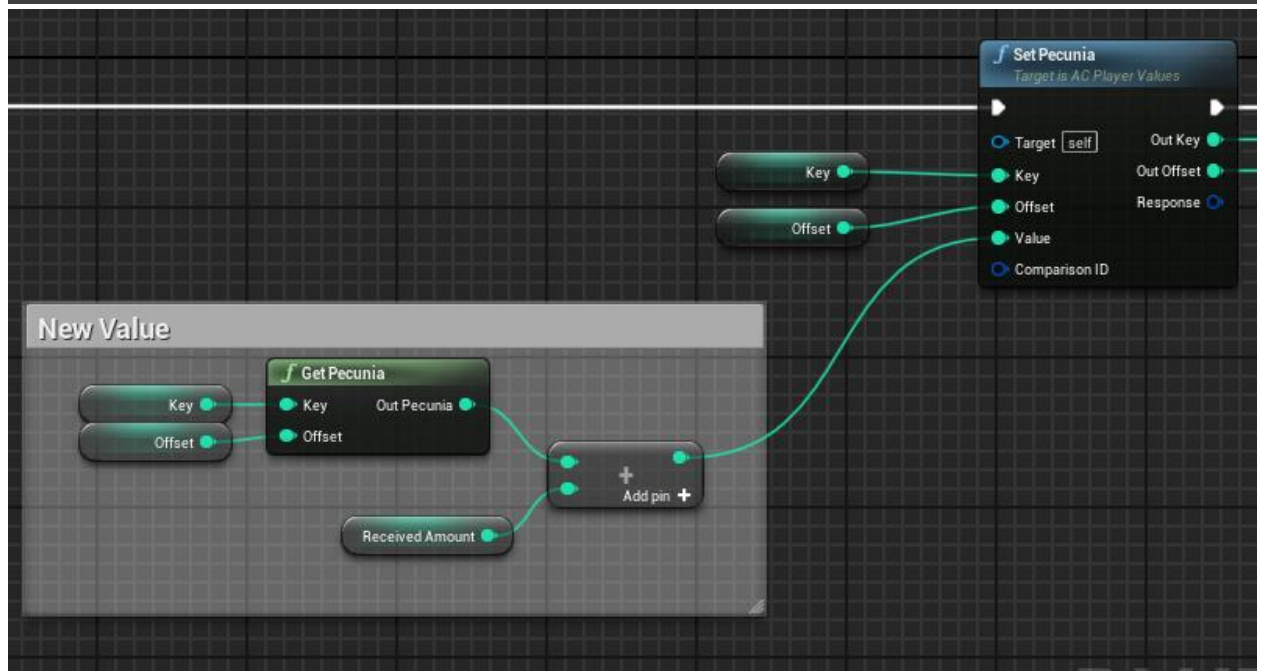
3.1 Security Control #1

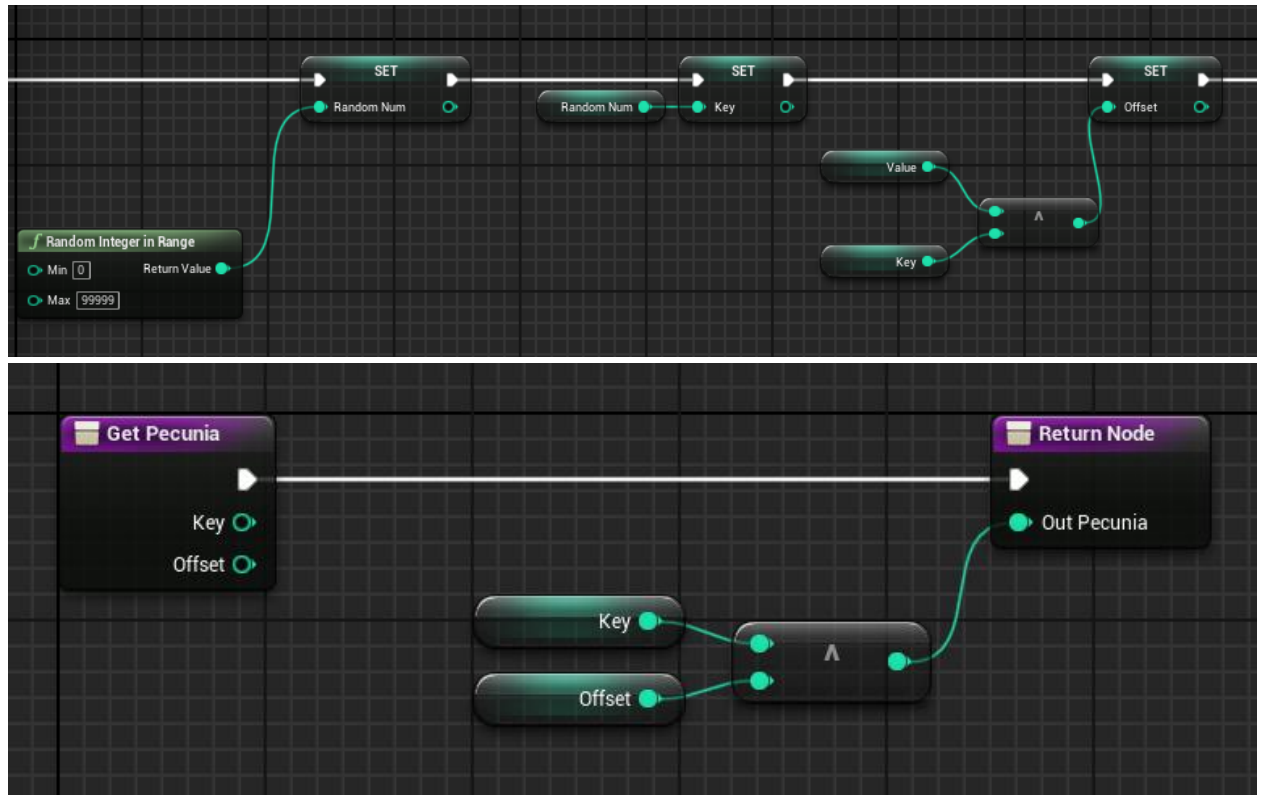
- **Name of Control:** Split Values with XOR Design
- **Threat Addressed:** Memory Editing or Runtime Cheating
- **CIA Principle:** Confidentiality & Integrity
- **Implementation Description:**
Currencies are split into two values which are the Key and the Offset. The real value of the Currency are never revealed.
- **Code Snippet:**

PlayerID	Guid				
Health	Float				
Sanity	Integer				
PecuniaKey	Integer				
PecuniaOffsetValue	Integer				

Default Values

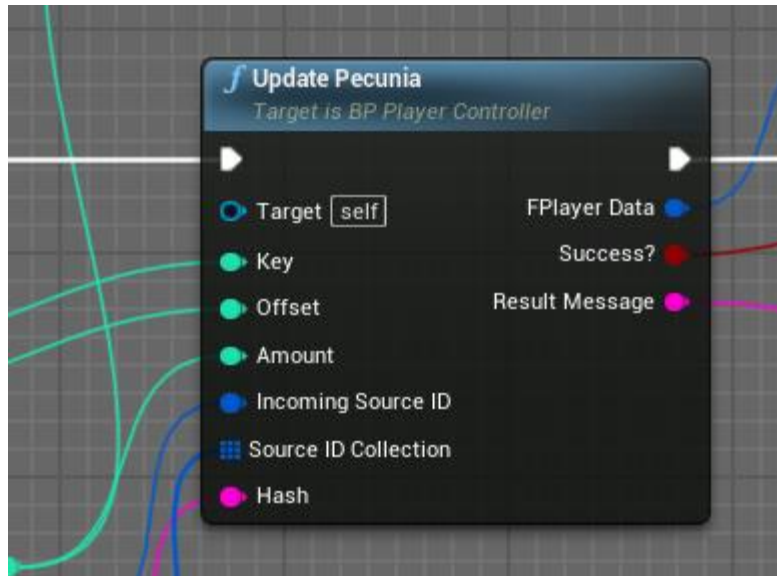
PlayerID	{00000000-0000-0000-0000-000000000000}
Health	0.0
Sanity	0
PecuniaKey	0
PecuniaOffsetValue	0





3.2 Security Control #2

- **Name of Control:** Source Ownership Design
- **Threat Addressed:** Unauthorized Resource Injection
- **CIA Principle:** Confidentiality & Availability
- **Implementation Description:**
Added Required Source Input whenever adding Currencies to the Player's Data to avoid unknown currency hacking.
- **Code Snippet:**



3.3 Security Control #3

- **Name of Control:** Hashing Save Files
- **Threat Addressed:** Save File Tampering
- **CIA Principle:** Integrity & Availability
- **Implementation Description:**
Adding File Signatures to Save Files to check the integrity of the Save File before loading it in the Game World. Avoids Save File Tampering.
- **Code Snippet:**

```

FString USecureHashing::GenerateSha256Hash(const FString& Input)
{
    FTCHARToUTF8 Convert(*Input);

    unsigned char Hash[SHA256_DIGEST_LENGTH];

    SHA256(
        reinterpret_cast<const unsigned char*>(Convert.Get()),
        Convert.Length(),
        Hash);

    FString Result;

    for (int32 i = 0; i < SHA256_DIGEST_LENGTH; i++)
    {
        Result += FString::Printf(
            TEXT("%02x"),
            Hash[i]);
    }

    return Result;
}

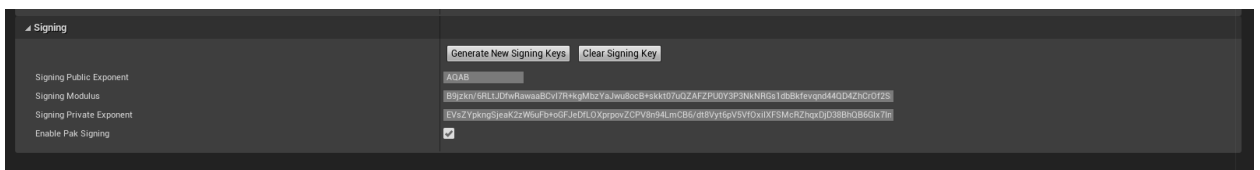
bool USecureHashing::VerifySha256Hash(const FString& Input, const FString&
ExistingHash)
{
    FString NewHash = GenerateSha256Hash(Input);

    return NewHash.Equals(
        ExistingHash,
        ESearchCase::IgnoreCase);
}

```

3.4 Security Control #3

- **Name of Control:** Pak Signing
- **Threat Addressed:** Asset replacement and malicious file modification
- **CIA Principle:** Confidentiality & Availability
- **Implementation Description:**
Enabling the Pak Signing along with generating an Signing Keys for encrypting the files into .pak format to avoid asset replacements.
- **Code Snippet:**



4. CIA Triad Application

4.1 Confidentiality

Runtime values are obfuscated using the Split Values with XOR design to reduce the display of sensitive in-memory data. Pak signing also helps to protect packaged game assets from unauthorized access and modification.

4.2 Integrity

Source Ownership with Hash Validation ensures that only the legitimate game systems can modify protected resources such as currency or rewards. Save file hashing also detects unauthorized modifications to player save data, while pak signing validates the authenticity and integrity of game assets.

4.3 Availability

The implemented security controls helps to maintain stable gameplay by preventing corrupted or tampered save files, invalid assets, and unauthorized modifications which may cause game system instability, crashes, or broken game functionalities.

5. Conclusion

This project demonstrated the importance of integrating security during early development stages of game development. By implementing memory obfuscation, hash validation, save file integrity checking, and pak signing, the game becomes more resistant to cheating, tampering, and unauthorized modifications. These security measures improve the game's reliability, fairness, and overall integrity of the system.